

# PRASHANT GUPTA

Backend Developer | Python | Django REST Framework | Docker | PostgreSQL  
guptaprashant931@gmail.com | 8103170556 | Bengaluru, Karnataka | [linkedin.com/in/prashant-gupta](https://www.linkedin.com/in/prashant-gupta) |  
[github.com/guptaprashant931-boop](https://github.com/guptaprashant931-boop)

## PROFESSIONAL SUMMARY

Backend Developer with hands-on experience building production-ready REST APIs using Python, Django, and Django REST Framework. Expert in designing and deploying async task pipelines with Celery and Redis, containerizing multi-service applications with Docker Compose, and implementing secure JWT-based authentication. Proficient in PostgreSQL schema design, ORM query optimization, and Git-based version control workflows.

## TECHNICAL SKILLS

|                   |                                                                                     |
|-------------------|-------------------------------------------------------------------------------------|
| <b>Languages:</b> | Python, JavaScript (ES6+)                                                           |
| <b>Backend:</b>   | Django, Django REST Framework, FastAPI, Celery, Celery Beat                         |
| <b>Databases:</b> | PostgreSQL, MySQL, Oracle SQL, Redis, ORM Query Optimization                        |
| <b>DevOps:</b>    | Docker, Docker Compose, Git, GitHub, Postman, Swagger UI                            |
| <b>Concepts:</b>  | REST API Design, JWT Authentication, Async Task Queues, OOP, MVC/MVT, Microservices |

## TRAINING

**Python Backend Developer** | PySpiders Institute, Bengaluru Jul 2025 – Feb 2026

- Developed 2 production-ready REST APIs (Blog API & Paragraph Word Frequency API) using Python, Django REST Framework, and PostgreSQL, delivering full CRUD functionality across 15+ endpoints
- Constructed an async word tokenization pipeline using Celery workers and Redis message broker, reducing API response time from 300ms to under 10ms for large text payloads
- Designed normalized PostgreSQL schemas with composite indexes, cutting word-frequency search query time by 60% compared to a full-table scan approach
- Enforced JWT authentication with access/refresh token rotation and refresh token blacklisting, securing 10+ API endpoints against unauthorized access
- Applied `select_related` and `prefetch_related` ORM optimizations across list endpoints, eliminating N+1 query issues and reducing database round-trips by up to 80%

## PROJECTS

### Paragraph Word Frequency API

[github.com/guptaprashant931-boop/Paragraph-Word-Frequency-API](https://github.com/guptaprashant931-boop/Paragraph-Word-Frequency-API)

Python | Django REST Framework | PostgreSQL | Celery | Redis | Docker Compose

- Built a REST API accepting multi-paragraph text (split on `\n\n`) and returning the top 10 paragraphs by word frequency via a single indexed SQL query, processing 1,000+ word inputs in under 10ms
- Architected normalized WordFrequency table with composite DB indexes, enabling  $O(\log n)$  search across millions of records without full-table scans
- Implemented Celery Beat periodic tasks for automatic reprocessing of failed paragraphs, achieving a 99%+ processing success rate even under worker crash scenarios
- Deployed full 5-service stack on Docker Compose with PostgreSQL health checks, automated database migrations on startup, and environment-based configuration via `python-decouple`

### Blog REST API

[github.com/guptaprashant931-boop/Blog\\_api](https://github.com/guptaprashant931-boop/Blog_api)

Python | Django | DRF | PostgreSQL | JWT | Docker | Swagger UI

- Engineered a nested comment system using a self-referential ForeignKey with a 2-level depth limit enforced in serializers; built atomic Like/Bookmark toggle endpoints using `get_or_create` with `unique_together` constraints
- Reduced list endpoint DB queries by 75% using `select_related` for ForeignKey joins and `prefetch_related` for ManyToMany tag relationships, eliminating N+1 query bottlenecks
- Implemented auto-slug generation via Django `pre_save` signals, multi-field filtering with `django-filter`, and full-text search across 5+ fields with paginated responses
- Auto-generated interactive Swagger API documentation using `drf-spectacular`; configured split settings (`base/dev/prod`) and Docker Compose for one-command local setup

## AI-Powered Support Ticket Management System

[github.com/guptaprashant931-boop/Support\\_ticket\\_management\\_system.git](https://github.com/guptaprashant931-boop/Support_ticket_management_system.git)

*Django REST Framework | React | PostgreSQL | Docker*

- Architected a full-stack ticket management system with 6+ REST APIs handling ticket creation, filtering, updates, and automated prioritization — reducing manual classification effort by ~60%.
- Implemented 4-level priority classification logic and multi-category routing for intelligent ticket dispatch.
- Designed 5+ React components using Hooks (useState, useEffect) for a dynamic, real-time UI experience.
- Modeled a normalized PostgreSQL schema with 6+ relational tables using Django ORM for efficient querying.
- Containerized frontend, backend, and database as independent Docker Compose services for reproducible deployment.

## EDUCATION

---

**B.Tech in Computer Science** Aug 2021 – Jun 2025

Hitkarini College of Engineering and Technology, Jabalpur, MP

## CERTIFICATIONS & COURSES

---

**Python Full Stack Development with Data Analytics** — PySpiders Institute, Bengaluru | 2025

**Docker & Containerization for Backend Developers** — Self-study via official Docker documentation | 2026

**Django REST Framework — Building APIs with Python** — Self-study via DRF official docs + projects | 2026